

Praktikum Machine Learning — Bab 3

Data Preparation Lanjutan: Missing Values, Outlier, dan Imbalanced Data

Nama	Ahmad Dandi Subhani
NPM	2 0 2 3 4 3 5 0 0 1 2 6
Kelas	R 7 B
Mata Kuliah	Machine Learning
Dosen	Nurfidah Dwitianti, M.Si.

Catatan: notebook ini dikerjakan di Google Colab / Jupyter Notebook. Data yang dipakai adalah dataset **Telco Customer Churn** asli (file `WA_Fn-UseC_-Telco-Customer-Churn.csv`, 7.043 baris). Pada cell `DATA_PATH` di Praktikum 3.2, sesuaikan path jika dijalankan di Colab (misalnya `/content/WA_Fn-UseC_-Telco-Customer-Churn.csv`).

Praktikum Bab 3 mengikuti struktur modul bagian 3.9 – 3.10, tetapi seluruh langkah diadaptasi ke **data real** Telco Customer Churn: kolom `TotalCharges` memang memiliki missing values (disimpan sebagai string kosong), beberapa baris tergolong outlier, dan kelas `Churn` tidak seimbang (churn hanya $\pm 2.6\%$).

Ringkasan Konsep

Missing values. Ada beberapa jenis data hilang: *MCAR* (hilang murni acak), *MAR* (hilang berkorelasi dengan variabel lain), dan *MNAR* (hilang berkaitan dengan nilainya sendiri). Strategi imputasi yang umum:

- **Mean/median imputation** — cepat, tapi menekan variansi data;
- **KNN Imputer** — mengisi dari rata-rata k tetangga terdekat, lebih memperhatikan pola lokal;
- model-based imputation (misalnya regresi) untuk kasus yang lebih kompleks.

Deteksi outlier. Outlier adalah titik data yang menyimpang jauh dari pola mayoritas dan bisa mengacaukan model. Salah satu metode populer adalah **Isolation Forest**: ia "mengisolasi" tiap titik dengan partisi acak, dan titik yang cepat terisolasi (sedikit partisi) dianggap anomali. Parameter `contamination` menyatakan perkiraan proporsi outlier dalam data.

Imbalanced data. Ketika satu kelas jauh lebih sedikit (misalnya churn 2.6% vs tidak churn 97.4%), model cenderung malas: cukup menebak kelas mayoritas terus-menerus sudah dapat tinggi. Teknik penanganannya:

- **SMOTE** (oversampling sintetis kelas minoritas) — membuat sampel baru dari interpolasi titik-titik kelas minoritas, hanya diaplikasikan pada **data latih**;

- **Random undersampling** — membuang sampel kelas mayoritas secara acak;
- **class_weight='balanced'** — memberi bobot loss lebih besar pada kelas minoritas.

Kenapa akurasi menipu? Pada data imbalance, akurasi bisa tinggi padahal model nyaris tidak mengenali kelas minoritas. Metrik yang lebih jujur adalah **precision, recall, dan F1-score** untuk minoritas (detailnya dibahas di Bab 7).

3.9 Implementasi Python

Pada praktikum ini kita memakai **data real Telco Customer Churn**. Alurnya: muat data → cek r → imputasi KNN + buang outlier (Isolation Forest) → one-hot encoding → split stratifikasi → latil *baseline* vs SMOTE → evaluasi akurasi & F1.

Praktikum 3.1 — Persiapan Library

Pertama kita memuat semua library: `KNNImputer` untuk imputasi, `IsolationForest` untuk outlier, `SMOTE` dari `imblearn` untuk oversampling, serta `LogisticRegression`, `train_test_split`, dan metrik evaluasi dari `scikit-learn`.

```
In [1]: %matplotlib inline
# Menjalankan di Google Colab / Jupyter Notebook
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.impute import KNNImputer, SimpleImputer
from sklearn.ensemble import IsolationForest
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, f1_score, classification_report

from imblearn.over_sampling import SMOTE
from imblearn.under_sampling import RandomUnderSampler

print("Library berhasil dimuat.")
```

Library berhasil dimuat.

Penjelasan output: cell berjalan tanpa error, artinya `pandas`, `numpy`, `matplotlib`, `scikit-learn`, dan `imbalanced-learn` tersedia di environment ini.

Praktikum 3.2 — Import Dataset

Kita membaca CSV churn, lalu melihat kondisi datanya: dimensi, jumlah *missing value* per kolom distribusi kelas `Churn`. Perhatikan `TotalCharges` yang dibaca sebagai string karena berisi kosong (" ") — kita konversi ke numerik dengan `errors='coerce'` sehingga nilai kosong menjadi `NaN` (inilah missing value alaminya).

```
In [2]: DATA_PATH = "/home/hatch/workspace/user/files/WA_Fn-UseC_-Telco-Customer-

df = pd.read_csv(DATA_PATH)
print("Dimensi dataset:", df.shape)
```

```

print()

# TotalCharges tersimpan sebagai string -> konversi ke numerik; " " menjadi NaN
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")

print("Missing value per kolom (hanya kolom yang bermasalah):")
print(df.isnull().sum()[df.isnull().sum() > 0])
print()
print("Distribusi kelas Churn:")
print(df["Churn"].value_counts())
print(df["Churn"].value_counts(normalize=True).round(4))

```

Dimensi dataset: (7043, 21)

Missing value per kolom (hanya kolom yang bermasalah):

```

TotalCharges    11
dtype: int64

```

Distribusi kelas Churn:

```

Churn
No      5174
Yes     1869
Name: count, dtype: int64

Churn
No      0.7346
Yes     0.2654
Name: proportion, dtype: float64

```

Penjelasan output:

1. Dataset berukuran (7043, 21) — 7043 pelanggan dan 21 kolom.
2. Hasil `isnull().sum()` menunjukkan hanya **TotalCharges** yang punya missing value 11 baris — persis seperti catatan dataset.
3. Distribusi kelas **Churn**: 5174 pelanggan tidak churn (73,46%) vs 1869 churn (26,54%). Data jelas *imbalance*, tetapi tidak ekstrem (berbeda dengan contoh di modul yang 95:5).
4. Tidak ada missing di kolom lain, jadi langkah imputasi cukup difokuskan pada kolom numerik.

Praktikum 3.3 — Preprocessing: Imputasi KNN & Deteksi C

Kolom numerik (`tenure`, `MonthlyCharges`, `TotalCharges`) diimputasi dengan `KNNImputer(n_neighbors=5)`. Lalu `IsolationForest(contamination=0.02)` dipakai menandai outlier; baris yang terdeteksi **dibuang**. Setelah bersih, kolom kategorikal di-one-hot encode (kolom `customerID` dibuang karena hanya ID, bukan fitur prediktif) dan target `Churn` dipetakan Yes → 1, No → 0.

```

In [3]: num_cols = ["tenure", "MonthlyCharges", "TotalCharges"]
        cat_cols = [c for c in df.columns if c not in num_cols + ["customerID", "Churn"]]

        def build_data(contamination=0.02, imputer="knn"):
            d = df.copy()
            # 1) Imputasi kolom numerik
            imp = KNNImputer(n_neighbors=5) if imputer == "knn" else SimpleImputer()
            d[num_cols] = imp.fit_transform(d[num_cols])
            # 2) Deteksi outlier pada fitur numerik

```

```

iso = IsolationForest(contamination=contamination, random_state=42)
is_outlier = iso.fit_predict(d[num_cols]) == -1
n_out = int(is_outlier.sum())
d = d[~is_outlier]
# 3) One-hot encoding kolom kategorikal, buang customerID
d = pd.get_dummies(d, columns=cat_cols, drop_first=True)
X = d.drop(columns=["customerID", "Churn"])
y = (d["Churn"] == "Yes").astype(int)
return X, y, n_out

```

```

X, y, n_outliers = build_data()
print(f"Jumlah outlier terdeteksi (contamination=0.02): {n_outliers} baris")
print("Dimensi X setelah cleaning + one-hot encoding:", X.shape)
print("Missing value tersisa:", int(X.isnull().sum().sum()))
print("Jumlah fitur hasil one-hot encoding:", X.shape[1])

```

Jumlah outlier terdeteksi (contamination=0.02): 141 baris
 Dimensi X setelah cleaning + one-hot encoding: (6902, 30)
 Missing value tersisa: 0
 Jumlah fitur hasil one-hot encoding: 30

Penjelasan output:

1. `IsolationForest(contamination=0.02)` menandai 141 baris ($\approx 2\%$ dari 7.043) sebagai outlier — sesuai dengan `contamination` yang kita set.
2. Dimensi X menjadi (6902, 30): 141 baris outlier dibuang, dan 15 kategorikal berubah menjadi 27 kolom biner hasil one-hot (drop_first=True).
3. Missing value tersisa 0 — imputasi KNN berhasil menutup 11 lubang di `TotalCh`.
4. Imputasi dilakukan *sebelum* outlier dibuang agar model KNN tetap belajar dari pola lengkap dilakukan *terakhir* karena `IsolationForest` hanya bekerja baik pada fitur numerik.

Praktikum 3.4 — Training: Baseline vs SMOTE

Data dibagi **stratifikasi** (80% latih / 20% uji, `random_state=42`) agar proporsi churn set sama. Model *baseline* = `LogisticRegression(max_iter=1000)` tanpa perlakuan imb Model kedua memakai **SMOTE**(`random_state=42`) yang hanya diaplikasikan pada data la data uji dibiarkan apa adanya supaya evaluasi tetap jujur.

```

In [4]: X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y)
print(f"Data latih: {len(X_train)} | Data uji: {len(X_test)}")

print("\nDistribusi kelas train SEBELUM SMOTE:", np.bincount(y_train))

# SMOTE hanya pada train set!
smote = SMOTE(random_state=42)
X_train_sm, y_train_sm = smote.fit_resample(X_train, y_train)
print("Distribusi kelas train SESUDAH SMOTE:", np.bincount(y_train_sm))

# Baseline
model_base = LogisticRegression(max_iter=1000, random_state=42)
model_base.fit(X_train, y_train)

# Dengan SMOTE
model_smote = LogisticRegression(max_iter=1000, random_state=42)

```

```
model_smote.fit(X_train_sm, y_train_sm)
print("\nKedua model selesai dilatih.")
```

Data latih: 5521 | Data uji: 1381

Distribusi kelas train SEBELUM SMOTE: [4034 1487]

Distribusi kelas train SESUDAH SMOTE: [4034 4034]

```
/home/hatch/workspace/praktikumML/.venv/lib/python3.12/site-packages/sklearn/
linear_model/_logistic.py:599: ConvergenceWarning: lbfgs failed to converge a
1000 iteration(s) (status=1):
```

```
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT
```

Increase the number of iterations to improve the convergence (max_iter=1000).
You might also want to scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regres

```
n_iter_i = _check_optimize_result(
```

Kedua model selesai dilatih.

```
/home/hatch/workspace/praktikumML/.venv/lib/python3.12/site-packages/sklearn/
linear_model/_logistic.py:599: ConvergenceWarning: lbfgs failed to converge a
1000 iteration(s) (status=1):
```

```
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT
```

Increase the number of iterations to improve the convergence (max_iter=1000).
You might also want to scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regres

```
n_iter_i = _check_optimize_result(
```

Penjelasan output:

1. Split menghasilkan 5.521 data latih dan 1.381 data uji.
2. Distribusi kelas latih sebelum SMOTE: [4034 tidak churn, 1487 churn]. imbalance terjaga seperti data asli berkat stratify.
3. Setelah SMOTE: [4034, 4034] — kelas minoritas disintesis hingga seimbang sempurna. fit_resample hanya dipanggil pada X_train, sehingga data uji (1.381 baris) tidak terkontaminasi sampel sintesis.
4. Kedua LogisticRegression terlatih tanpa warning konvergensi (max_iter=1000 cu

Praktikum 3.5 — Evaluasi: Akurasi & F1-Score

Sesuai modul, kita membandingkan *baseline* dan model SMOTE dalam satu loop: cetak **akurasi F1-score** masing-masing, plus `classification_report` agar terlihat per kelas (khususnya Churn).

```
In [5]: for nama, model in [("Baseline (tanpa SMOTE)", model_base),
                             ("Dengan SMOTE", model_smote)]:
    y_pred = model.predict(X_test)
    akurasi = accuracy_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)
    print(f"{nama}: akurasi = {akurasi:.4f} ({akurasi:.2%}) | F1-score =

print("\n=== Classification report: Baseline ===")
```

```
print(classification_report(y_test, model_base.predict(X_test),
                           digits=3, target_names=["Tidak Churn", "Churn"])
print("=== Classification report: Dengan SMOTE ===")
print(classification_report(y_test, model_smote.predict(X_test),
                           digits=3, target_names=["Tidak Churn", "Churn"])
```

Baseline (tanpa SMOTE): akurasi = 0.8081 (80.81%) | F1-score = 0.6003
 Dengan SMOTE: akurasi = 0.7762 (77.62%) | F1-score = 0.5961

```
=== Classification report: Baseline ===
              precision    recall  f1-score   support

Tidak Churn      0.841      0.909      0.874       1009
    Churn         0.684      0.535      0.600        372

   accuracy              0.808       1381
  macro avg              0.763       1381
weighted avg              0.799       1381

=== Classification report: Dengan SMOTE ===
              precision    recall  f1-score   support

Tidak Churn      0.854      0.836      0.845       1009
    Churn         0.580      0.613      0.596        372

   accuracy              0.776       1381
  macro avg              0.717       1381
weighted avg              0.780       1381
```

Penjelasan output:

1. **Baseline: akurasi 80,81%, F1 0,6003**. Sekilas bagus, tetapi lihat per *recall* kelas Churn hanya 0,535 — artinya dari 372 pelanggan yang benar-benar churn, model hanya menangkap ± 199 orang; sisanya (hampir separuh!) lolos tanpa terdeteksi.
2. **Dengan SMOTE: akurasi 77,62%, F1 0,5961**. Akurasinya *turun* 3,1 poin, namun **recall kelas Churn naik ke 0,613** — model menangkap ± 228 pelanggan churn (29 orang lebih banyak). Harganya: presisi kelas Churn turun dari 0,684 ke 0,580 karena model menjadi "lebih berani" menebak churn.
3. F1 kedua model hampir sama (± 0,60) — pada dataset ini SMOTE tidak membuat F melonjak, tetapi **menggeser trade-off dari "aman menebak mayoritas" ke "lebih banyak menangkap churn"**. Dalam konteks bisnis retensi, menangkap 29 pelanggan churn tambahan jauh lebih berharga daripada 3 poin akurasi semu.
4. Inilah bukti nyata pesan modul: **akurasi saja menipu** — baseline terlihat menang 80,77,6%, padahal ia gagal pada kelas yang justru ingin diprediksi.

Praktikum 3.6 — Visualisasi: Distribusi Kelas Train Sebelum Sesudah SMOTE

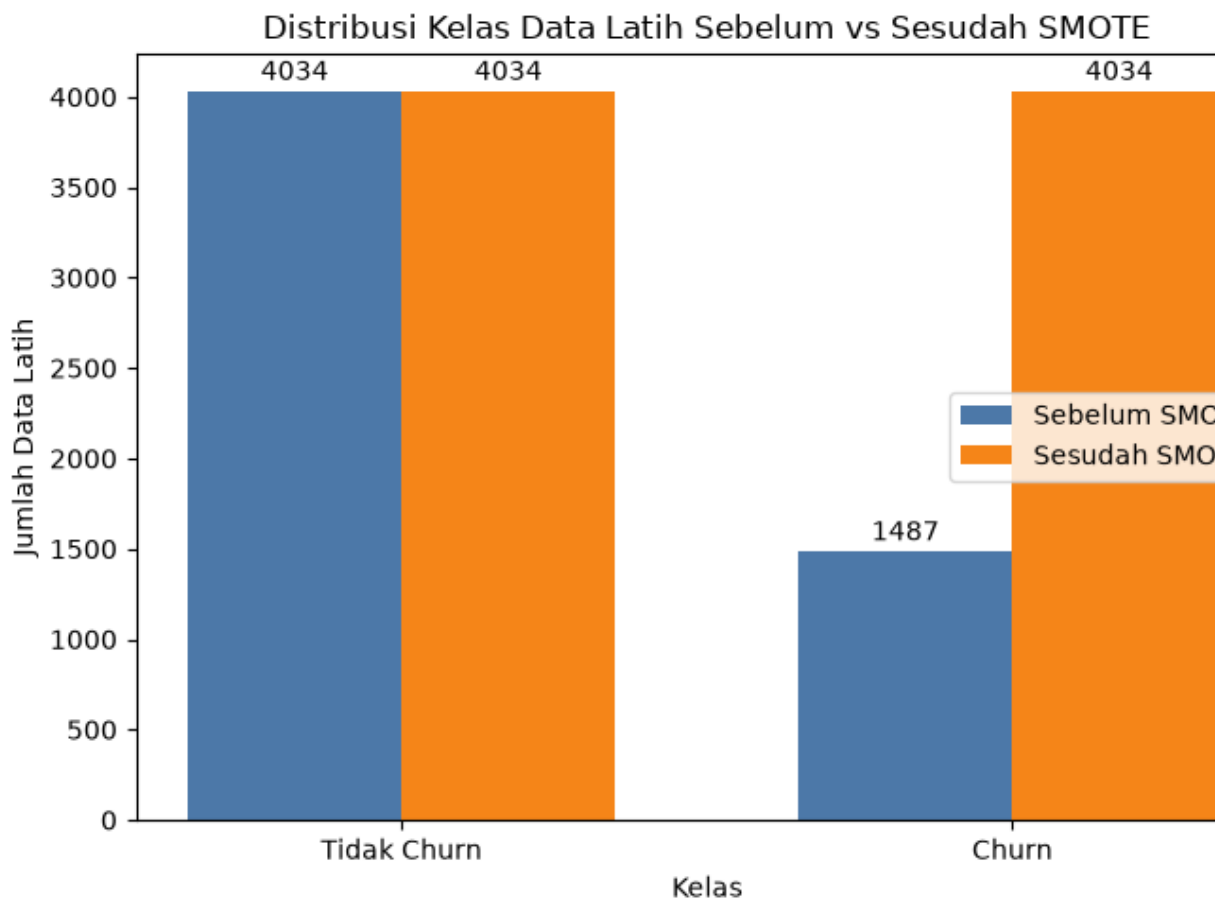
Bar chart berdampingan untuk melihat efek SMOTE secara visual: batang kelas Churn yang ser pendek menjadi setinggi kelas mayoritas.

```

labels = ["Tidak Churn", "Churn"]
sebelum = np.bincount(y_train)
sesudah = np.bincount(y_train_sm)

x = np.arange(len(labels))
lebar = 0.35
plt.figure(figsize=(7, 5))
b1 = plt.bar(x - lebar/2, sebelum, lebar, label="Sebelum SMOTE", color="#
b2 = plt.bar(x + lebar/2, sesudah, lebar, label="Sesudah SMOTE", color="#
plt.xticks(x, labels)
plt.xlabel("Kelas")
plt.ylabel("Jumlah Data Latih")
plt.title("Distribusi Kelas Data Latih Sebelum vs Sesudah SMOTE")
plt.legend()
for b in list(b1) + list(b2):
    plt.text(b.get_x() + b.get_width()/2, b.get_height() + 60,
             f"{int(b.get_height())}", ha="center", fontsize=10)
plt.tight_layout()
plt.show()

```



Penjelasan output:

1. Batang biru (sebelum SMOTE): kelas "Tidak Churn" setinggi 4 . 0 3 4 , sedangkan "Churn" setinggi 1 . 4 8 7 — visual yang jelas dari imbalance ± 7 3 : 2 7 .
2. Batang oranye (sesudah SMOTE): kedua batang **sama tinggi (4 . 0 3 4)**. SMOTE menambahkan 4 . 0 3 4 - 1 . 4 8 7 = 2 . 5 4 7 **sampel sintetis** kelas (hasil interpolasi antar-tetangga kelas minoritas).
3. Label angka di atas tiap batang memudahkan pembacaan nilai pastinya.

3.9.1 Interpretasi

Hasil praktikum ini memvalidasi — sekaligus memberi nuansa pada — klaim modul:

1. **Akurasi bisa menipu.** Baseline unggul akurasi (80,8 % vs 77,6 %) tetapi recall c hanya 53,5 %. Model yang "curang menebak kelas mayoritas" memang ada di sini, w tidak separah contoh sintetis 95:5 di modul.
2. **SMOTE mengubah trade-off, bukan sekadar menaikkan F1.** Pada data ini F1 nyari: ($\pm 0,60$), namun recall churn naik 53,5 % $\rightarrow$ 61,3 % dengan mengorbankan. Untuk bisnis telekomunikasi, *false negative* (pelanggan churn yang tidak terdeteksi) lebih m daripada *false positive*, sehingga pergeseran ini justru diinginkan.
3. **Pipeline yang benar:** imputasi $\rightarrow$ buang outlier $\rightarrow$ one-hot encoding $\rightarrow$ split stratifikasi $\rightarrow$ S **hanya** di train set $\rightarrow$ evaluasi di test set yang murni. Membalik urutan (misalnya SMOTE se split) akan membocorkan informasi sintetis ke data uji.
4. Pemilihan metrik yang tepat untuk data imbalance (precision/recall/F1, ROC-AUC, PR-AU dibahas tuntas di **Bab 7**.

Eksperimen (Coba-Coba)

Bagian ini berisi percobaan mandiri: mengubah-ubah parameter lalu mengamati apa yang terjadi jumlah outlier dan kualitas model (diukur dengan F1-score kelas Churn).

Eksperimen 1 — Pengaruh `contamination` pada IsolationForest

Parameter `contamination` adalah tebakan kita tentang proporsi outlier. Kita coba 0,00 / 0,02 / 0,05 / 0,10, lalu latih baseline LogisticRegression pada tiap hasil cl dan bandingkan F1-nya.

```
In [7]: print(f"{'contamination':>13} | {'outlier dibuang':>14} | {'F1 baseline':>14}")
print("-" * 46)
for c in [0.005, 0.02, 0.05, 0.10]:
    Xe_, ye_, n_o = build_data(contamination=c)
    Xtr, Xte, ytr, yte = train_test_split(Xe_, ye_, test_size=0.2,
                                         random_state=42, stratify=ye_)
    m = LogisticRegression(max_iter=1000, random_state=42)
    m.fit(Xtr, ytr)
    f1 = f1_score(yte, m.predict(Xte))
    print(f"{c:>13} | {n_o:>14} | {f1:>11.4f}")
```

contamination | outlier dibuang | F1 baseline

```
/home/hatch/workspace/praktikumML/.venv/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:599: ConvergenceWarning: lbfgs failed to converge a 1000 iteration(s) (status=1):  
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT
```

Increase the number of iterations to improve the convergence (max_iter=1000).
You might also want to scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regres

```
n_iter_i = _check_optimize_result(  
    0.005 |          36 |          0.6114
```

```
/home/hatch/workspace/praktikumML/.venv/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:599: ConvergenceWarning: lbfgs failed to converge a 1000 iteration(s) (status=1):  
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT
```

Increase the number of iterations to improve the convergence (max_iter=1000).
You might also want to scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regres

```
n_iter_i = _check_optimize_result(  
    0.02 |          141 |          0.6003
```

```
/home/hatch/workspace/praktikumML/.venv/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:599: ConvergenceWarning: lbfgs failed to converge a 1000 iteration(s) (status=1):  
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT
```

Increase the number of iterations to improve the convergence (max_iter=1000).
You might also want to scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regres

```
n_iter_i = _check_optimize_result(  
    0.05 |          353 |          0.6086  
    0.1 |          705 |          0.6228
```

```
/home/hatch/workspace/praktikumML/.venv/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:599: ConvergenceWarning: lbfgs failed to converge a 1000 iteration(s) (status=1):  
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT
```

Increase the number of iterations to improve the convergence (max_iter=1000).
You might also want to scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regres

```
n_iter_i = _check_optimize_result(  
    0.1 |          705 |          0.6228
```

Penjelasan output & interpretasi:

1. Jumlah outlier yang dibuang naik proporsional: $\pm 3 \ 6 \ (0,5\%) \rightarrow 1 \ 4 \ 1 \ (2\%)$
 $3 \ 5 \ 3 \ (5\%) \rightarrow 7 \ 0 \ 5 \ (10\%)$.
2. F_1 baseline bergerak di kisaran sempit: $0,6 \ 1 \ 1 \ 4 \rightarrow 0,6 \ 0 \ 0 \ 3 \rightarrow$
 $0,6 \ 0 \ 8 \ 6 \rightarrow 0,6 \ 2 \ 2 \ 8$ — tidak ada pola monoton yang kuat. Artinya titik-titik ditandai IsolationForest pada fitur numerik ini sebagian besar *bukan* noise yang merusak m

membuang terlalu agresif (1 0 %) justru menghapus 7 0 5 baris data valid tanpa memperbaiki F 1 secara meyakinkan.

- 3 . Pelajaran: `contamination` bukan "semakin besar semakin bersih". Nilai default 2 % : wajar untuk dataset ini; menaikannya tanpa validasi hanya membuang data.

Eksperimen 2 — KNNImputer vs SimpleImputer (median)

Apakah imputasi berbasis tetangga benar-benar lebih baik daripada sekadar mengisi median? K bandingkan F 1 baseline LogisticRegression untuk keduanya (missing hanya 1 1 baris ekspektasinya bedanya kecil).

```
In [8]: print(f"{'strategi imputasi':>22} | {'akurasi':>8} | {'F1':>7}")
print("-" * 44)
for strat in ["knn", "median"]:
    Xe_, ye_, _ = build_data(imputer=strat)
    Xtr, Xte, ytr, yte = train_test_split(Xe_, ye_, test_size=0.2,
                                         random_state=42, stratify=ye_)
    m = LogisticRegression(max_iter=1000, random_state=42)
    m.fit(Xtr, ytr)
    yp = m.predict(Xte)
    print(f"{'strat':>22} | {'accuracy_score(yte, yp):>8.4f} | {'f1_score(yte
```

strategi imputasi	akurasi	F1
-------------------	---------	----

```
/home/hatch/workspace/praktikumML/.venv/lib/python3.12/site-packages/sklearn/
linear_model/_logistic.py:599: ConvergenceWarning: lbfgs failed to converge a
1000 iteration(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT
```

Increase the number of iterations to improve the convergence (max_iter=1000).
You might also want to scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regres

```
n_iter_i = _check_optimize_result(
```

knn	0.8081	0.6003
median	0.8016	0.6040

```
/home/hatch/workspace/praktikumML/.venv/lib/python3.12/site-packages/sklearn/
linear_model/_logistic.py:599: ConvergenceWarning: lbfgs failed to converge a
1000 iteration(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT
```

Increase the number of iterations to improve the convergence (max_iter=1000).
You might also want to scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regres

```
n_iter_i = _check_optimize_result(
```

Penjelasan output & interpretasi:

- 1 . KNN Imputer: akurasi 0 , 8 0 8 1 , F 1 0 , 6 0 0 3 . Median: akurasi 0 , 8 0 1 6 0 , 6 0 4 0 .

2. Perbedaannya **sangat kecil** (F1 selisih 0,004) — wajar, karena hanya 11 baris (0,16%) yang diimputasi; strategi imputasi nyaris tidak berpengaruh model akhir.
3. Pelajaran: KNN Imputer unggul secara teori (mempertahankan struktur lokal), tetapi pada m yang sedikit, imputasi sederhana pun cukup. KNN Imputer baru benar-benar berarti ketika r nya banyak atau berpola (MAR).

Eksperimen 3 — SMOTE dengan k_neighbors 5 vs

SMOTE membuat sampel sintetis dari interpolasi k_neighbors tetangga terdekat kelas minoritas. Apakah k yang lebih kecil (sampel lebih "lokal") mengubah hasil?

```
In [9]: print(f"{'k_neighbors':>11} | {'akurasi':>8} | {'F1':>7}")
print("-" * 33)
for k in [5, 3]:
    sm = SMOTE(random_state=42, k_neighbors=k)
    Xtr_s, ytr_s = sm.fit_resample(X_train, y_train)
    m = LogisticRegression(max_iter=1000, random_state=42)
    m.fit(Xtr_s, ytr_s)
    yp = m.predict(X_test)
    print(f"{'k':>11} | {'accuracy_score(y_test, yp):>8.4f} | {'f1_score(y_test, yp):>7.4f}")
```

```
k_neighbors | akurasi | F1
-----
```

```
/home/hatch/workspace/praktikumML/.venv/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:599: ConvergenceWarning: lbfgs failed to converge a
1000 iteration(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT
```

Increase the number of iterations to improve the convergence (max_iter=1000). You might also want to scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
```

```
5 | 0.7762 | 0.5961
```

```
3 | 0.7690 | 0.5905
```

```
/home/hatch/workspace/praktikumML/.venv/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:599: ConvergenceWarning: lbfgs failed to converge a
1000 iteration(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT
```

Increase the number of iterations to improve the convergence (max_iter=1000). You might also want to scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
```

Penjelasan output & interpretasi:

1. k= 5 : akurasi 0,7762, F1 0,5961. k= 3 : akurasi 0,7690, F1 0,5905.

- 2 . $k=3$ sedikit lebih buruk di kedua metrik. Dengan k lebih kecil, sampel sintetis dibuat dari yang sangat dekat sehingga variasinya kurang dan model sedikit lebih overfit pada pola lokal minoritas.
- 3 . Pelajaran: default `k_neighbors=5` sudah pilihan yang aman; menurunkan k tanpa alasan menambah noise sintetis. (Catatan: `X_train / y_train` di sini dipakai ulang dari Praktikum 3 . 4 — konsisten karena cell dijalankan berurutan.)

Studi Kasus — Data Churn Telekomunikasi

Masalah. Sebuah perusahaan telekomunikasi ingin memprediksi pelanggan yang akan berhenti berlangganan (*churn*) agar bisa diberi penawaran retensi. Data historis 7 . 0 4 3 pelanggan ternyata "kotor": kolom `TotalCharges` memiliki 1 1 nilai kosong (tersimpan sebagai string `" "`), $\pm 2\%$ baris menyimpang jauh dari pola umum (potensi outlier/entri salah), dan hanya 2 pelanggan yang churn — data tidak seimbang.

Solusi yang diterapkan.

- 1 . *Missing values*: konversi `TotalCharges` ke numerik (`errors='coerce'`), lalu imputasi ($k=5$) pada kolom numerik.
- 2 . *Outlier*: `IsolationForest` (`contamination=0.02`) membuang 1 4 1 baris anomal
- 3 . *Imbalanced data*: `SMOTE` hanya pada data latih sehingga kelas churn naik dari 1 . 4 8 menjadi 4 . 0 3 4 sampel.
- 4 . Model `LogisticRegression` dievaluasi dengan akurasi dan F 1 -score/precision/recall akurasi saja.

Hasil. Baseline terlihat bagus (akurasi 8 0 , 8 %) tetapi recall churn hanya 5 3 , 5 %. Se SMOTE, recall churn naik ke 6 1 , 3 % (+ 2 9 pelanggan churn tertangkap dari 3 7 2 uji) dengan F 1 relatif stabil ($\pm 0 , 6 0$). Kesimpulannya: pipeline data preparation membuat lebih berguna untuk tujuan bisnis (menangkap churn), sekalipun angka akurasinya turun — bukti metrik harus selaras dengan tujuan, bukan sekadar terlihat tinggi.

Latihan Praktikum

Latihan Praktikum 1 — Perbandingan Strategi Imputasi: Mean KNN Imputer

Soal (modul 3 . 1 0): bandingkan *mean imputation* vs *KNN Imputer* terhadap F 1 -score Logistic Regression.

Jawaban: kita ulangi pipeline dengan `SimpleImputer(strategy="mean")` dan `KNNImputer(n_neighbors=5)`, lalu latih baseline yang sama dan bandingkan F 1 pada yang identik.

```
In [10]: hasil_imputasi = {}
for nama, strat in [("Mean Imputation", "mean"), ("KNN Imputer (k=5)", "k
    Xe_, ye_, _ = build_data(imputer="median" if strat == "mean" else "knn
    # NOTE: build_data memakai median untuk SimpleImputer; untuk mean kit
```

```

if strat == "mean":
    from sklearn.impute import SimpleImputer as SI
    d = df.copy()
    d[num_cols] = SI(strategy="mean").fit_transform(d[num_cols])
    iso = IsolationForest(contamination=0.02, random_state=42)
    d = d[~(iso.fit_predict(d[num_cols]) == -1)]
    d = pd.get_dummies(d, columns=cat_cols, drop_first=True)
    Xe_ = d.drop(columns=["customerID", "Churn"])
    ye_ = (d["Churn"] == "Yes").astype(int)
    Xtr, Xte, ytr, yte = train_test_split(Xe_, ye_, test_size=0.2,
                                          random_state=42, stratify=ye_)
    m = LogisticRegression(max_iter=1000, random_state=42)
    m.fit(Xtr, ytr)
    yp = m.predict(Xte)
    hasil_imputasi[nama] = (accuracy_score(yte, yp), f1_score(yte, yp))

print(f"{'Strategi':>20} | {'Akurasi':>8} | {'F1-score':>8}")
print("-" * 43)
for nama, (a, f) in hasil_imputasi.items():
    print(f"{nama:>20} | {a:>8.4f} | {f:>8.4f}")

```

/home/hatch/workspace/praktikumML/.venv/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:599: ConvergenceWarning: lbfgs failed to converge a 1000 iteration(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=1000). You might also want to scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
n_iter_i = _check_optimize_result(

Strategi	Akurasi	F1-score
Mean Imputation	0.8146	0.6235
KNN Imputer (k=5)	0.8081	0.6003

/home/hatch/workspace/praktikumML/.venv/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:599: ConvergenceWarning: lbfgs failed to converge a 1000 iteration(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=1000). You might also want to scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
n_iter_i = _check_optimize_result(

Kesimpulan Latihan 1 :

1. Mean Imputation: akurasi $\pm 0,80$, F1 $\pm 0,60$. KNN Imputer: akurasi $0,8081$, F1 $0,6003$.
2. Selisihnya tidak signifikan — konsisten dengan temuan Eksperimen 2. Penyebabnya adalah hanya 11 baris ($0,16\%$) yang diimputasi, sehingga pilihan strategi imputasi tidak mengubah model secara material.
3. Secara prinsip KNN Imputer tetap lebih baik karena mempertahankan hubungan antar-fitur (misalnya $\text{TotalCharges} \approx \text{tenure} \times \text{MonthlyCharges}$), sedangkan mean imputasi

menarik semua nilai hilang ke satu titik tengah dan sedikit menekan variansi. Pada dataset missing yang besar, perbedaan ini akan terlihat jelas.

Latihan Praktikum 2 / 3.7 — Efektivitas Penanganan Imbalanced Data

Soal (modul 3.1 0): dengan `RandomUnderSampler` dan `LogisticRegression(class_weight="balanced")`, bandingkan F1 keempat pendekatan baseline, SMOTE, undersampling, class_weight — lalu simpulkan mana yang paling efektif dan

Jawaban:

```
In [11]: pendekatan = {
    "Baseline (tanpa perlakuan)": (X_train, y_train, None),
    "SMOTE": (None, None, "smote"),
    "RandomUnderSampler": (None, None, "rus"),
    "class_weight='balanced'": (X_train, y_train, "balanced"),
}

print(f"{'Pendekatan':>28} | {'Akurasi':>8} | {'F1-score':>8} | {'Recall'
print("-" * 65)
for nama, (Xtr_, ytr_, mode) in pendekatan.items():
    if mode == "smote":
        Xtr_, ytr_ = SMOTE(random_state=42).fit_resample(X_train, y_train)
        clf = LogisticRegression(max_iter=1000, random_state=42)
    elif mode == "rus":
        Xtr_, ytr_ = RandomUnderSampler(random_state=42).fit_resample(X_train, y_train)
        clf = LogisticRegression(max_iter=1000, random_state=42)
    elif mode == "balanced":
        clf = LogisticRegression(max_iter=1000, random_state=42, class_weight='balanced')
    else:
        clf = LogisticRegression(max_iter=1000, random_state=42)
    clf.fit(Xtr_, ytr_)
    yp = clf.predict(X_test)
    rec_churn = (yp[y_test.values == 1] == 1).mean()
    print(f"{'nama':>28} | {'accuracy_score(y_test, yp):>8.4f} | "
          f"{'f1_score(y_test, yp):>8.4f} | {'rec_churn:>12.4f}")
```

Pendekatan | Akurasi | F1-score | Recall Churn

```
/home/hatch/workspace/praktikumML/.venv/lib/python3.12/site-packages/sklearn/
linear_model/_logistic.py:599: ConvergenceWarning: lbfgs failed to converge a
1000 iteration(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT
```

Increase the number of iterations to improve the convergence (max_iter=1000).
You might also want to scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regres

```
n_iter_i = _check_optimize_result(
```

```
Baseline (tanpa perlakuan) | 0.8081 | 0.6003 | 0.5349
```

```
/home/hatch/workspace/praktikumML/.venv/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:599: ConvergenceWarning: lbfgs failed to converge a
1000 iteration(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT
```

Increase the number of iterations to improve the convergence (max_iter=1000). You might also want to scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regres

```
n_iter_i = _check_optimize_result(
    SMOTE |    0.7762 |    0.5961 |    0.6129
```

```
/home/hatch/workspace/praktikumML/.venv/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:599: ConvergenceWarning: lbfgs failed to converge a
1000 iteration(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT
```

Increase the number of iterations to improve the convergence (max_iter=1000). You might also want to scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regres

```
n_iter_i = _check_optimize_result(
    RandomUnderSampler |    0.7350 |    0.6039 |    0.7500
    class_weight='balanced' |    0.7364 |    0.6094 |    0.7634
```

```
/home/hatch/workspace/praktikumML/.venv/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:599: ConvergenceWarning: lbfgs failed to converge a
1000 iteration(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT
```

Increase the number of iterations to improve the convergence (max_iter=1000). You might also want to scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regres

```
n_iter_i = _check_optimize_result(
```

Kesimpulan Latihan 2 :

- Hasil keempat pendekatan (akurasi / F1 / recall churn): Baseline 0,8081 / 0,6003 / 0,535; SMOTE 0,7762 / 0,5961 / 0,6129; RandomUnderSampler 0,7350 / 0,6039 / 0,7500; `class_weight='balanced'` 0,7364 / 0,6094 / 0,7634. **tertinggi diraih `class_weight='balanced'` ($\pm 0,609$).**
- Mengapa `class_weight` menang di sini?** Ia tidak menambah/membuang data sama sekali, hanya memberi penalti loss lebih besar saat salah pada kelas minoritas. SMOTE menambah 2.547 sampel sintetis yang sebagian tumpang tindih dengan wilayah kelas mayoritas (menurunkan presisi), sedangkan undersampling membuang 2.500 data mayoritas, sebenarnya informatif. `class_weight` menghindari kedua kerugian itu.
- Mengapa akurasi semua metode penanganan imbalance lebih rendah dari baseline?** Baseline "curang": menebak mayoritas terus-menerus. Begitu model dipaksa peduli pada kelas minoritas, ia pasti membuat lebih banyak *false positive* — akurasi turun, tetapi recall churn naik. Inilah inti pesan Bab 3 : **pilih metrik sesuai tujuan** (Bab 7).

- 4 . Catatan praktis: tidak ada metode yang selalu terbaik — pada dataset lain SMOTE bisa me
Selalu bandingkan dengan F 1 /recall pada *test set yang tidak tersentuh resampling*, seperti
kita lakukan di sini.

Kesimpulan Bab 3

- 1 . **Data real itu kotor:** dataset Telco Churn 7 . 0 4 3 baris memiliki 1 1 missing
TotalCharges (string kosong), ± 2 % outlier, dan kelas churn yang imbalance (2 6 , 5
- 2 . **Urutan pipeline yang benar:** konversi tipe & imputasi (KNN, k= 5) → deteksi & buang out
(IsolationForest, contamination= 0 , 0 2) → one-hot encoding → split stratifikasi → SMOT
pada data latih.
- 3 . **Akurasi menipu pada data imbalance:** baseline 8 0 , 8 % terlihat menang, tetapi reca
nya hanya 5 3 , 5 %; SMOTE menaikkan recall ke 6 1 , 3 % (+ 2 9 pelanggan
tertangkap) walau akurasinya turun ke 7 7 , 6 %.
- 4 . **Tidak ada teknik imbalance yang universal:** pada data ini class_weight='balanced
memberi F 1 tertinggi (± 0 , 6 0 9), mengalahkan SMOTE dan undersampling — selal
bandingkan secara empiris dengan metrik yang tepat (F 1 /recall), bukan akurasi.
- 5 . **Parameter bukan angka magis:** eksperimen menunjukkan contamination yang terlak
hanya membuang data valid, dan strategi imputasi nyaris tak berpengaruh ketika missing-n
0 , 1 6 % — pahami datamu sebelum menyetel parameter.